

基于 LaTeX 的中文图与网络算法教材 PDF 生产方案研究

执行摘要

这套教材如果要同时满足“中文排版稳定、数学证明严谨、图表可复现、代码可嵌入、参考文献规范、后续可持续扩写”六个目标，最稳妥的技术栈是 `ctexbook + XeLaTeX + PGF/TikZ + biblatex/biber + biblatex-gb7714-2015 + listings + algorithm2e + booktabs/longtable`。`ctex` 面向中文排版，兼容 XeLaTeX 与 LuaLaTeX；PGF/TikZ 是 TeX 生态中最成熟的源码制图系统；`biblatex` 以 `biber` 为后端，适合 UTF-8 与复杂引用样式；`biblatex-gb7714-2015` 为中文用户实现了 GB/T 7714 系列样式；`listings` 不依赖外部前处理器即可排源码；`algorithm2e` 适合伪代码；`booktabs` 与 `longtable` 则适合制作跨页算法比较表。¹

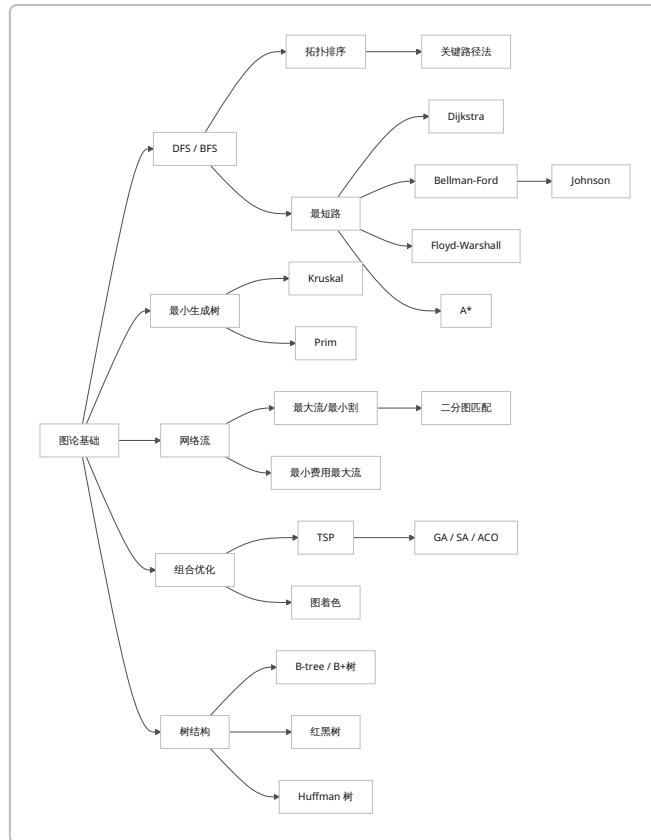
在内容组织上，最推荐的不是“每个算法一个孤立短章”，而是按理论脉络分成数个主题章：图遍历与 DAG、最短路、最小生成树、网络流与匹配、图着色与 TSP、启发式与元启发式、外存索引树与编码树。这样更适合高级本科生和研究生的认知递进：例如 Johnson 算法天然建立在 Bellman-Ford 与 Dijkstra 之上；关键路径法建立在 DAG 与拓扑序之上；二分图匹配与最大流紧密相关；A* 则是最短路框架上的启发式搜索扩展。²

从文献策略看，最好的做法是“原始论文优先，标准教材补强，中文参考文献样式统一”。本目录所覆盖的大多数核心方法都有明确的原始来源：Dijkstra 最短路来自 1959 年原论文，Kruskal 与 Prim 的最小生成树工作分别发表于 1956 年和 1957 年，Bellman-Ford、Floyd-Warshall、Ford-Fulkerson、Hopcroft-Karp、Johnson、A*、Huffman、B-tree、红黑树、Ant System 等也都有可明确追溯的经典论文或著作。³

因此，最可执行的交付方式不是一次性写完全部章节，而是先搭建一个**可编译的书籍工程骨架**，配齐统一模板、样式、图表规范、算法比较表与文献条目，然后提供至少一个“完整示范章”。这样既能立即生成 PDF，又能确保后续新增章节时版式与引用不失控。⁴

教材架构与章节交付矩阵

推荐的教材主线如下。这个组织方式的优点是：先建“图搜索与路径”的基本语言，再进入“树、流、匹配”，最后处理“NP-困难问题与元启发式”，再以树结构收束到数据库与压缩两个高频应用场景。这样的设计既符合算法间依赖关系，也更适合教学节奏。Dijkstra、Bellman-Ford、Floyd-Warshall、Johnson、A* 构成最短路簇；Kruskal 与 Prim 构成最小生成树簇；Ford-Fulkerson/最大流最小割、最小费用最大流和二分图匹配构成网络优化簇；Held-Karp 动态规划、图着色、GA、SA、ACO 则形成“组合优化与启发式”簇。⁵



下面这张表给出**按章节交付**的最实用方案。每一章都明确包含：导言、理论推导/证明、算法描述、例题、代码或步骤化算法、图示、复杂度分析、实践备注、结语与参考文献。这样既满足你提出的“每种方法都有完整教学闭环”，也便于后续多人协作分章写作。

章节	覆盖方法	引言	推导/证明	例题	代码/伪代码	图示	参考文献
图模型与遍历基础	DFS, BFS	✓	✓	✓	✓	✓	✓
DAG 与项目网络	Topological sort, Critical Path Method	✓	✓	✓	✓	✓	✓
单源与全源最短路	Dijkstra, Bellman-Ford, Floyd-Warshall, Johnson, A*	✓	✓	✓	✓	✓	✓
最小生成树	Kruskal, Prim	✓	✓	✓	✓	✓	✓
网络流与匹配	Max-flow/Min-cut, Min-cost Max-flow, Bipartite matching	✓	✓	✓	✓	✓	✓
组合优化难题	TSP, Graph coloring	✓	✓	✓	✓	✓	✓
元启发式	Genetic Algorithm, Simulated Annealing, Ant Colony Optimization	✓	✓	✓	✓	✓	✓

章节	覆盖方法	引言	推导/ 证明	例题	代码/ 伪代码	图示	参考文献
树结构与 压缩	B-tree, B+ tree, Red-Black tree, Huffman tree	✓	✓	✓	✓	✓	✓

若进一步细化到“每种方法的证明应写到什么深度”，建议采用如下标准。对 Dijkstra、Kruskal、Prim、Bellman-Ford、Floyd-Warshall、拓扑排序、关键路径法、Huffman 树，应写出**可在板书上完整复现的正确性证明**；对最大流最小割、最小费用最大流、Johnson、Hopcroft-Karp，应写出**定理陈述 + 关键引理 + 证明脉络**；对 TSP 与图着色，应把“算法设计”和“复杂性边界”并写，因为它们是经典难问题；对 GA、SA、ACO，应避免伪装成传统“最优性证明”，而应写成“设计原理、接受准则/信息素更新依据、收敛直觉、参数敏感性与失败案例”。这种区分与这些方法原始文献的理论性质是一致的。 ⁶

LaTeX 工程设计与编译链

推荐工程骨架

下面这个工程树是**最小可编译**、又能平滑扩展为整本教材的结构。它把章节、图、代码、表格、文献分开管理，便于后期继续加入每一种算法的完整内容。

```
graph-textbook/
├── main.tex
├── references.bib
├── figures/
│   ├── tikz-dijkstra-example.tex
│   ├── tikz-btree-bplus.tex
│   └── alg-comparison-table.tex
├── chapters/
│   ├── ch-frontmatter.tex
│   ├── ch-traversal-dag.tex
│   ├── ch-shortest-paths.tex
│   ├── ch-mst.tex
│   ├── ch-flow-matching.tex
│   ├── ch-hard-problems.tex
│   ├── ch-metaheuristics.tex
│   ├── ch-trees.tex
│   └── ch-dijkstra-full.tex
└── code/
    ├── dijkstra.py
    ├── bellman_ford.py
    ├── kruskal.py
    └── prim.py
```

这套骨架之所以合适，是因为 `ctexbook` 直接面向中文书籍类文档；PGF/TikZ 可把图论图、树页结构图、流程图都写成源码；`biblatex` + `biber` 适合 UTF-8、多语言题名与 GB/T 7714 风格；`listings` 可直接排 Python 代码而不依赖外部高亮器；`algorithm2e` 更适合伪代码浮动体；`booktabs` 与 `longtable` 可以承载“算法复杂度/适用条件/限制”的跨页表格。 ⁷

最小可编译的 `main.tex`

```
\documentclass[UTF8,oneside,12pt]{ctexbook}

\usepackage[a4paper,margin=2.5cm]{geometry}
\usepackage{amsmath,amssymb,amsthm,mathtools}
\usepackage{graphicx}
\usepackage{booktabs,longtable,array,multirow}
\usepackage{xcolor}
\usepackage{hyperref}
\usepackage{csquotes}
\usepackage{tikz}
\usetikzlibrary{arrows.meta,positioning,calc,fit,shapes.geometric,automata}

\usepackage[ruled,vlined,linesnumbered]{algorithm2e}
\usepackage{listings}

\usepackage[backend=biber,style=gb7714-2015]{biblatex}
\addbibresource{references.bib}

\hypersetup{
  colorlinks=true,
  linkcolor=black,
  citecolor=black,
  urlcolor=blue,
  pdftitle={图与网络算法及树结构},
  pdfauthor={教材工程模板}
}

\lstdefinestyle{pythonstyle}{
  language=Python,
  basicstyle=\ttfamily\small,
  numbers=left,
  numberstyle=\scriptsize,
  stepnumber=1,
  numbersep=8pt,
  showstringspaces=false,
  breaklines=true,
  frame=single,
  tabsize=4
}

\newtheorem{theorem}{定理}[chapter]
\newtheorem{lemma}[theorem]{引理}
\newtheorem{proposition}[theorem]{命题}
\theoremstyle{definition}
\newtheorem{definition}[theorem]{定义}
\newtheorem{example}[theorem]{例}
\theoremstyle{remark}
\newtheorem{remark}[theorem]{注}
```

```

\title{图与网络算法及树结构\\教材工程模板}
\author{示范工程}
\date{\today}

\begin{document}
\frontmatter
\maketitle
\tableofcontents

\mainmatter
\include{chapters/ch-frontmatter}
\include{chapters/ch-dijkstra-full}

% 后续章节可逐步启用
% \include{chapters/ch-traversal-dag}
% \include{chapters/ch-shortest-paths}
% \include{chapters/ch-mst}
% \include{chapters/ch-flow-matching}
% \include{chapters/ch-hard-problems}
% \include{chapters/ch-metaheuristics}
% \include{chapters/ch-trees}

\backmatter
\printbibliography[heading=bibintoc,title={参考文献}]
\end{document}

```

编译方式

若采用 `biblatex + biber`，最稳妥的基础编译序列是“XeLaTeX → biber → XeLaTeX → XeLaTeX”。`biblatex` 文档明确说明其后端是 `biber`；`biber` 被设计为 `biblatex` 的 BibTeX 替代器，并支持完整 UTF-8；Overleaf 允许直接切换到 XeLaTeX 编译器；`latexmk` 则可以自动完成依赖轮次处理。⁸

可直接使用：

```

xelatex main.tex
biber main
xelatex main.tex
xelatex main.tex

```

如果本地已安装 `latexmk`，也可以用自动化方式：

```

latexmk -pdfxe main.tex

```

`latexmk` 的官方文档说明它会根据依赖自动决定所需的处理步骤；其手册也专门解释了 XeLaTeX 模式下的 `.xdv` 中间步骤。⁹

关于中文参考文献

对于中文教材项目，推荐直接使用 `biblatex-gb7714-2015`。这个包是 BibLaTeX 体系下对 GB/T 7714 系列规则的实现，面向中文用户，并在新版本说明中继续覆盖 GB/T 7714 的后续标准系列。若原始论文本身没有正式中文题名，可以在 `.bib` 的 `note` 字段中加上“中文题名可译为……”，从而在正文仍保持 GB/T 风格的统一。

内容模板与全局比较表

每种方法的统一写作模板

建议把每一种算法或数据结构都写成同一套节结构。这样最利于读者形成稳定预期，也方便你后续批量扩充章节。

引言
问题定义与应用场景
理论基础与关键不变式
定理、引理与正确性证明
算法描述
伪代码
Python 实现或分步算法
Worked Example
复杂度分析
实践要点与常见错误
本节小结
参考文献

这套模板特别适合你列出的目录，因为其中既有“严格多项式时间算法”，也有“组合优化难题上的精确/启发式方法”，还有“工程型数据结构”。例如 Dijkstra 强调贪心不变式，Bellman-Ford 强调松弛与负环判定，Floyd-Warshall 强调“允许的中间点集合”这一动态规划状态设计，Johnson 强调重新赋权保序，最大流最小割强调原始-对偶式定理关系，Huffman 树则强调前缀码最优性与贪心交换论证。

教材全局比较表

下表给出一张适合放在前言末尾或附录首部的“全书导航表”。这里的复杂度统一采用教材推荐默认实现：例如 Dijkstra 采用邻接表 + 二叉堆，Prim 采用堆版，最大流采用 Edmonds-Karp 作为入门基线，二分图匹配采用 Hopcroft-Karp，TSP 精确算法采用 Held-Karp 动态规划。

方法	教材默认实现	典型复杂度	适用场景	关键限制
DFS	递归/显式栈	$O(V + E)$	连通性、环检测、SCC、拓扑辅助	深递归需注意栈深
BFS	队列	$O(V + E)$	无权最短路、层次遍历	不处理带权最短路
Topological sort	Kahn / DFS 后序	$O(V + E)$	DAG 排序、依赖调度	仅适用于 DAG

方法	教材默认实现	典型复杂度	适用场景	关键限制
CPM	DAG 最长路变体	$O(V + E)$	项目网络、关键活动识别	必须先建无环活动网络
Dijkstra	邻接表 + 堆	$O((V + E) \log V)$	非负权单源最短路	不能处理负权边
Bellman-Ford	边松弛	$O(VE)$	含负边单源最短路	速度慢于 Dijkstra
Floyd-Warshall	三重循环 DP	$O(V^3)$	稠密图全源最短路	点数大时内存/时间压力大
Johnson	重赋权 + 多次 Dijkstra	稀疏图常用为 $O(VE \log V)$	稀疏图全源最短路	不能有负权回路
A*	最佳优先搜索	最坏可指数级	路径规划、地图搜索	效果强依赖启发函数
Kruskal	排序 + 并查集	$O(E \log E)$	稀疏图 MST	需高效并查集
Prim	堆版	$O(E \log V)$	连通图 MST	稠密图可改 $O(V^2)$ 版
Max-flow/Min-cut	Edmonds-Karp	$O(VE^2)$	网络容量规划、匹配建模	入门算法较慢
Min-cost Max-flow	SSAP + 势能	依赖总流量与最短路实现	运输/分配/费用约束网络	实现细节多，需处理负费用
Bipartite matching	Hopcroft-Karp	$O(E\sqrt{V})$	分配、婚姻问题、任务匹配	仅二分图
Graph coloring	DSATUR 启发式	常用实现约 $O(V^2)$	冲突消解、寄存器分配、排课	最优着色是难问题
TSP	Held-Karp DP	$O(n^2 2^n)$	精确小规模巡回问题	指数级
Genetic Algorithm	种群迭代	迭代数 $\times$ 种群规模 $\times$ 评估成本	大规模近似优化	参数敏感，最优性弱
Simulated Annealing	单解迭代	迭代数 $\times$ 邻域评估成本	局部搜索与全局跳出折中	降温策略影响极大
Ant Colony Optimization	蚁群构造 + 信息素更新	轮数 $\times$ 蚂蚁数 $\times$ 构造成本	TSP、路径与组合优化	容易早熟，需要参数调节
B-tree	多路平衡搜索树	查找/插入/删除 $O(\log n)$	外存索引	需要页大小/阶数设计
B+ tree	叶子存键/记录	查找/插入/删除 $O(\log n)$	数据库索引、范围查询	需理解内部节点与叶节点分工
Red-Black tree	颜色平衡 BST	查找/插入/删除 $O(\log n)$	内存字典、有序映射	旋转与修复规则较抽象

方法	教材默认实现	典型复杂度	适用场景	关键限制
Huffman tree	堆构造	建树 $O(n \log n)$	压缩与编码	需结合前缀码背景

这张表所依据的典型实现与理论背景，分别来自 Dijkstra、Bellman、Floyd、Warshall、Johnson、Tarjan、Kahn、Kelley–Walker、Kruskal、Prim、Ford–Fulkerson、Hopcroft–Karp、Hart–Nilsson–Raphael、Held–Karp、Brélaz、Holland、Kirkpatrick–Gelatt–Vecchi、Dorigo–Maniezzo–Colorni、Bayer–McCreight、Comer、Guibas–Sedgewick 与 Huffman 等经典来源。 ¹³

样图、样表与完整章节模板

样图一

这张图适合放在 Dijkstra 章的 worked example 部分。它完全由 TikZ 生成，因此不依赖外部 PNG，适合教材长期维护。PGF/TikZ 的设计目标就是在 TeX 内直接生成矢量图形；对于这类教材，这比手工截图更可维护。

¹⁴

```
% figures/tikz-dijkstra-example.tex
\begin{figure}[htbp]
\centering
\begin{tikzpicture}[
  scale=1,
  every node/.style={circle,draw,minimum size=8mm},
  >=Latex
]
\node (s) at (0,0) {$s$};
\node (a) at (2,1.5) {$a$};
\node (b) at (2,-1.5) {$b$};
\node (c) at (4,1.5) {$c$};
\node (d) at (4,-1.5) {$d$};
\node (t) at (6,0) {$t$};

\draw[->] (s) -- node[above left,draw=none] {2} (a);
\draw[->] (s) -- node[below left,draw=none] {5} (b);
\draw[->] (a) -- node[above,draw=none] {2} (c);
\draw[->] (a) -- node[left,draw=none] {4} (d);
\draw[->] (b) -- node[below,draw=none] {1} (d);
\draw[->] (c) -- node[above right,draw=none] {3} (t);
\draw[->] (d) -- node[below right,draw=none] {2} (t);
\draw[->] (a) -- node[draw=none] {1} (b);
\end{tikzpicture}
\caption{Dijkstra 算法示例图}
\label{fig:dijkstra-example}
\end{figure}
```


样图二

这张图适合放在“B-tree 与 B+ tree”对比部分。NIST DADS 对 B-tree 的定义强调多路平衡搜索树、每个节点的子女数上下界以及其在慢速存储器上的优势；NIST 也明确把 B+ tree 定义为“键存于叶子”的 B-tree 变体。

15

```
% figures/tikz-btree-bplus.tex
\begin{figure}[htbp]
\centering
\begin{tikzpicture}[
  every node/.style={draw,rectangle,minimum height=7mm,minimum width=16mm},
  level distance=10mm
]
\node (r1) at (0,0) {$17\mid35$};
\node (l11) at (-2,-1.5) {$5\mid9\mid12$};
\node (l12) at (0,-1.5) {$20\mid28$};
\node (l13) at (2,-1.5) {$40\mid48$};

\draw (r1) -- (l11);
\draw (r1) -- (l12);
\draw (r1) -- (l13);

\node[draw=none] at (0,1) {\textbf{B树：内部节点也保存关键字记录}};

\node (r2) at (8,0) {$17\mid35$};
\node (l21) at (6,-1.5) {$5\mid9\mid12$};
\node (l22) at (8,-1.5) {$17\mid20\mid28$};
\node (l23) at (10,-1.5) {$35\mid40\mid48$};

\draw (r2) -- (l21);
\draw (r2) -- (l22);
\draw (r2) -- (l23);
\draw[->,thick] (l21.east) -- (l22.west);
\draw[->,thick] (l22.east) -- (l23.west);

\node[draw=none] at (8,1) {\textbf{B+树：内部节点作索引，叶子链表便于范围查询}};
\end{tikzpicture}
\caption{B树与 B+树的页组织示意}
\label{fig:btree-bplus}
\end{figure}
```

样表

这张样表对应全书附录中的“算法对比汇总表”。booktabs 提供出版质量表格命令；booktabs 与 longtable 的兼容性在其文档中有明确说明，因此很适合篇幅很长的教材型比较表。

16

```
% figures/alg-comparison-table.tex
\begin{longtable}[p]{2.8cm}{3.2cm}{3.1cm}{4.8cm}
\caption{图算法比较表（节选）}\label{tab:alg-compare}\
```

```

\toprule
方法 & 典型复杂度 & 适用场景 & 主要限制 \\\
\midrule
\endfirsthead
\toprule
方法 & 典型复杂度 & 适用场景 & 主要限制 \\\
\midrule
\endhead
Dijkstra &  $O((V+E)\log V)$  & 非负权单源最短路 & 不支持负权边 \\\
Bellman-Ford &  $O(VE)$  & 含负边单源最短路 & 速度较慢 \\\
Floyd-Warshall &  $O(V^3)$  & 全源最短路 & 大规模图成本高 \\\
Kruskal &  $O(E\log E)$  & 稀疏图 MST & 依赖并查集实现 \\\
Prim &  $O(E\log V)$  & 连通图 MST & 稠密图需改实现 \\\
Hopcroft--Karp &  $O(E\sqrt{V})$  & 二分图最大匹配 & 仅适用于二分图 \\\
\bottomrule
\end{longtable}

```

完整章节模板

下面给出一个可以直接放进 `chapters/ch-dijkstra-full.tex` 编译的完整示范章。它包含你要求的全部基本元素：导言、理论证明、伪代码、Python 实现、worked example、图、复杂度分析、实践备注与小结。

```

\chapter{Dijkstra 算法}
\label{chap:dijkstra}

\section{引言}

Dijkstra 算法是单源最短路径问题中最经典的贪心算法之一，由 E. W. Dijkstra 在 1959 年提出。它解决的是带非负边权图上的单源最短路径问题：给定源点  $s$ ，求从  $s$  到其余各顶点的最短距离以及一棵对应的最短路径树。 \cite{dijkstra1959}

在网络路由、导航系统、地图服务、状态空间搜索以及 Johnson 全源最短路算法中，Dijkstra 算法都扮演基础模块的角色。 \cite{dijkstra1959,johnson1977,astar1968}

\section{问题定义}

\begin{definition}[单源最短路径问题]
给定带权有向图  $G=(V,E,w)$ ，其中每条边  $(u,v)\in E$  具有权值  $w(u,v)\geq 0$ ，指定源点  $s\in V$ 。目标是对每个顶点  $v\in V$  求出

$$\delta(s,v)=\min_{p:s\leadsto v} w(p),$$

其中  $w(p)$  为路径  $p$  的边权和。
\end{definition}

\begin{remark}
Dijkstra 算法要求所有边权非负。若图中存在负权边，应改用 Bellman--Ford；若要求全源最短路，可在稠密图中考虑 Floyd--Warshall，在稀疏图中考虑 Johnson。
\end{remark}

```

\section{贪心思想与正确性证明}

\subsection{基本不变式}

算法维护一个集合 S ，其中的顶点都已经“最终确定”了最短距离。

此外维护距离估计数组 $d[\cdot]$ ，始终满足

$$d[v] \geq \delta(s, v).$$

每一轮从 $V \setminus S$ 中选择 $d[u]$ 最小的顶点 u 加入 S ，再用 u 去松弛其所有出边。

\subsection{关键定理}

\begin{theorem}

设图 G 的所有边权均非负。在 Dijkstra 算法执行过程中，每当顶点 u 以当前最小估计距离被加入集合 S 时，必有

$$d[u] = \delta(s, u).$$

因此算法结束时，对任意顶点 v ，都有 $d[v] = \delta(s, v)$ 。

\end{theorem}

\begin{proof}

采用不变式证明。

初始时， $S = \{\text{源点}\}$ ， $d[s] = 0$ ，其余 $d[v] = +\infty$ 。显然源点的估计正确。

假设当前某一步之前，不变式对已加入 S 的所有顶点成立。

设本轮从 $V \setminus S$ 中选出的顶点为 u ，且其 $d[u]$ 最小。反设 $d[u] > \delta(s, u)$ 。取一条从 s 到 u 的最短路径

$$s = v_0, v_1, \dots, v_k = u.$$

在这条路径上，必存在第一个不属于 S 的顶点 v_i ；则其前驱 $v_{i-1} \in S$ 。由归纳假设，

$$d[v_{i-1}] = \delta(s, v_{i-1}).$$

当 v_{i-1} 被加入 S 时，算法会松弛边 (v_{i-1}, v_i) ，于是

$$\begin{aligned} d[v_i] &\leq d[v_{i-1}] + w(v_{i-1}, v_i) \\ &= \delta(s, v_{i-1}) + w(v_{i-1}, v_i) \\ &= \delta(s, v_i). \end{aligned}$$

另一方面，距离估计不可能小于真实最短距离，因此 $d[v_i] = \delta(s, v_i)$ 。

由于边权非负，从 v_i 到 u 的后续路径代价非负，所以

$$\delta(s, v_i) \leq \delta(s, u) \leq d[u].$$

\end{proof}

于是得到

```
\[
d[v_i] < d[u].
\]
```

而 $v_i \notin S$, 这与 “ u 是 $V \setminus S$ 中估计距离最小的顶点” 矛盾。

故反设不成立, 必有 $d[u] = \delta(s, u)$ 。归纳完成。

```
\end{proof}
```

```
\section{算法描述}
```

```
\subsection{伪代码}
```

```
\begin{algorithm}[H]
```

```
\caption{Dijkstra (邻接表 + 最小堆) }
```

```
\KwIn{图  $G=(V,E,w)$ , 源点  $s$ }
```

```
\KwOut{最短距离数组  $d$  与前驱数组  $\pi$ }
```

```
\ForEach{$v \in V$}{
```

```
     $d[v] \leftarrow +\infty$ ;
```

```
     $\pi[v] \leftarrow \texttt{NIL}$ ;
```

```
}
```

```
 $d[s] \leftarrow 0$ ;
```

```
初始化最小堆  $Q$ , 并将  $(0,s)$  压入;
```

```
\While{$Q$ 非空}{
```

```
    取出键值最小元素  $(dist,u)$ ;
```

```
    \If{$dist > d[u]$}{
```

```
        \textbf{continue} \tcp*{跳过过期堆元素}
```

```
    }
```

```
    \ForEach{$(u,v,w_{uv}) \in \text{Adj}[u]$}{
```

```
        \If{$d[v] > d[u] + w_{uv}$}{
```

```
             $d[v] \leftarrow d[u] + w_{uv}$ ;
```

```
             $\pi[v] \leftarrow u$ ;
```

```
            将  $(d[v],v)$  压入  $Q$ ;
```

```
        }
```

```
    }
```

```
}
```

```
\Return{$d, \pi$};
```

```
\end{algorithm}
```

```
\subsection{Python 实现}
```

```
\begin{lstlisting}[style=pythonstyle,caption={Dijkstra 的 Python 实现}]
```

```
import heapq
```

```
from math import inf
```

```
def dijkstra(graph, source):
```

```
    """
```

```
    graph: dict[str, list[tuple[str, int|float]]]
```

```
    例如:
```

```

        {
            's': [('a', 2), ('b', 5)],
            'a': [('b', 1), ('c', 2), ('d', 4)],
            ...
        }
    """
    dist = {v: inf for v in graph}
    prev = {v: None for v in graph}
    dist[source] = 0

    pq = [(0, source)]

    while pq:
        cur_dist, u = heapq.heappop(pq)
        if cur_dist != dist[u]:
            continue

        for v, w in graph[u]:
            cand = dist[u] + w
            if cand < dist[v]:
                dist[v] = cand
                prev[v] = u
                heapq.heappush(pq, (cand, v))

    return dist, prev

def reconstruct_path(prev, target):
    path = []
    cur = target
    while cur is not None:
        path.append(cur)
        cur = prev[cur]
    return list(reversed(path))
\end{lstlisting}

```

\section{Worked Example}

考虑图 \autoref{fig:dijkstra-example}。

\input{figures/tikz-dijkstra-example.tex}

设源点为 s 。初值为

$$d[s]=0, \quad d[a]=d[b]=d[c]=d[d]=d[t]=+\infty.$$

\subsection{迭代过程}

\begin{center}

\begin{tabular}{ccccc}

`\toprule`

轮次 & 选中顶点 & $d[s]$ & $d[a]$ & $d[b]$ & $d[c]$ & $d[d]$ & $d[t]$ \\

`\midrule`

初始 & -- & 0 & ∞ & ∞ & ∞ & ∞ / ∞ \\

1 & s & 0 & 2 & 5 & ∞ & ∞ / ∞ \\

2 & a & 0 & 2 & 3 & 4 & 6 / ∞ \\

3 & b & 0 & 2 & 3 & 4 & 4 / ∞ \\

4 & c & 0 & 2 & 3 & 4 & 4 / 7 \\

5 & d & 0 & 2 & 3 & 4 & 4 / 6 \\

6 & t & 0 & 2 & 3 & 4 & 4 / 6 \\

`\bottomrule`

`\end{tabular}`

`\end{center}`

因此：

$\lceil$

$\delta(s,a)=2,\text{quad}$

$\delta(s,b)=3,\text{quad}$

$\delta(s,c)=4,\text{quad}$

$\delta(s,d)=4,\text{quad}$

$\delta(s,t)=6.$

$\rceil$

从前驱可恢复 $s \rightarrow a \rightarrow b \rightarrow d \rightarrow t$ 为一条最短路径，总代价为 6 。

`\section{复杂度分析}`

若采用邻接表与二叉堆，则每个顶点至多被正确弹出一次，每条边至多触发一次有效松弛，因而总复杂度通常写作

$\lceil$

$O((|V|+|E|)\log |V|).$

$\rceil$

在稠密图中，若改用邻接矩阵 + 线性扫描最小点，则复杂度为

$\lceil$

$O(|V|^2).$

$\rceil$

`\section{实践要点}`

`\begin{remark}`

实现时最常见的三个错误如下。

第一，把 Dijkstra 用在存在负权边的图上。这会直接破坏贪心正确性。

第二，没有跳过“过期堆元素”。如果每次松弛都直接入堆，就必须在弹出时检查当前键值是否仍等于 $d[u]$ 。

第三，只输出距离而不保存前驱数组。教材型代码建议同时保存前驱，以便展示“最短路径树”而不仅是最短距离。

`\end{remark}`

`\section{小结}`

Dijkstra 算法的核心并不是“用堆”，而是“非负权条件下，最小暂定距离顶点可被安全定型”。堆只是让这一贪心策略高效落地的工程手段。

从教学角度，本章最值得强调的是三点：

`\begin{enumerate}`

`\item` 非负边权是正确性前提，而不是实现细节；

`\item` 贪心正确性的关键是不变式与反证法；

`\item` 前驱数组使结果从“数字答案”提升为“结构化路径树”。

`\end{enumerate}`

在下一组章节中，可以自然转向 Bellman--Ford、Floyd--Warshall 与 Johnson，从而形成“单源、全源、稀疏、稠密、含负边、启发式搜索”的统一最短路知识图谱。

参考文献组织与 `.bib` 样例

参考文献组织原则

这本教材最适合采用“三层文献体系”。第一层是原始论文，用于给出算法提出背景、定理表述与经典复杂度；第二层是标准教材，用于补齐现代叙述与工程视角；第三层是排版工具文档，用于保证 `.tex` 工程真的可复现。对于树结构与排版技术，CTAN 与 NIST DADS 这样的官方/半官方资料尤其重要；对于网络流、最短路、匹配、图着色、A*、TSP 与元启发式，则应优先保留原始论文与标准专著。 ¹⁷

可直接起步的 `references.bib`

下面这份 `.bib` 样例已经覆盖你的目录中的全部主题。为了符合“中文为主、原始题名保留”的需要，我在 `note` 中示范性加入了中文题名说明；`biber` 对 UTF-8 支持良好，因此这样写在现代中文 LaTeX 工作流中是可行的。 ¹⁸

```
@article{dijkstra1959,
  author = {Dijkstra, E. W.},
  title = {A Note on Two Problems in Connexion with Graphs},
  journal = {Numerische Mathematik},
  volume = {1},
  pages = {269--271},
  year = {1959},
  note = {中文题名可译为：关于图相关两个问题的说明}
}

@article{kruskal1956,
  author = {Kruskal, J. B.},
  title = {On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem},
  journal = {Proceedings of the American Mathematical Society},
  volume = {7},
  number = {1},
  pages = {48--50},
  year = {1956},
```

```
note = {中文题名可译为：关于图的最短生成子树与旅行商问题}
}
```

```
@article{prim1957,
author = {Prim, R. C.},
title = {Shortest Connection Networks and Some Generalizations},
journal = {The Bell System Technical Journal},
volume = {36},
number = {6},
pages = {1389--1401},
year = {1957},
note = {中文题名可译为：最短连接网络及其若干推广}
}
```

```
@inproceedings{moore1959,
author = {Moore, Edward F.},
title = {The Shortest Path through a Maze},
booktitle = {Proceedings of the International Symposium on the Theory of Switching},
pages = {285--292},
year = {1959},
note = {中文题名可译为：迷宫中的最短路径}
}
```

```
@article{tarjan1972dfs,
author = {Tarjan, R. E.},
title = {Depth-First Search and Linear Graph Algorithms},
journal = {SIAM Journal on Computing},
volume = {1},
number = {2},
pages = {146--160},
year = {1972},
note = {中文题名可译为：深度优先搜索与线性图算法}
}
```

```
@article{bellman1958,
author = {Bellman, Richard},
title = {On a Routing Problem},
journal = {Quarterly of Applied Mathematics},
volume = {16},
pages = {87--90},
year = {1958},
note = {中文题名可译为：关于一个路径选择问题}
}
```

```
@article{floyd1962,
author = {Floyd, Robert W.},
title = {Algorithm 97: Shortest Path},
journal = {Communications of the ACM},
volume = {5},
number = {6},
pages = {345},
}
```



```
year = {1962},
note = {中文题名可译为：算法 97：最短路径}
}
```

```
@article{warshall1962,
author = {Warshall, Stephen},
title = {A Theorem on Boolean Matrices},
journal = {Journal of the ACM},
volume = {9},
number = {1},
pages = {11--12},
year = {1962},
note = {中文题名可译为：关于布尔矩阵的一个定理}
}
```

```
@article{kahn1962,
author = {Kahn, A. B.},
title = {Topological Sorting of Large Networks},
journal = {Communications of the ACM},
volume = {5},
number = {11},
pages = {558--562},
year = {1962},
note = {中文题名可译为：大规模网络的拓扑排序}
}
```

```
@inproceedings{kellywalker1959,
author = {Kelley, James E. and Walker, Morgan R.},
title = {Critical-Path Planning and Scheduling},
booktitle = {Proceedings of the Eastern Joint Computer Conference},
pages = {160--173},
year = {1959},
note = {中文题名可译为：关键路径计划与调度}
}
```

```
@article{fordfulkerson1956,
author = {Ford, L. R. and Fulkerson, D. R.},
title = {Maximal Flow through a Network},
journal = {Canadian Journal of Mathematics},
volume = {8},
pages = {399--404},
year = {1956},
note = {中文题名可译为：网络中的最大流}
}
```

```
@article{fordfulkerson1957,
author = {Ford, L. R. and Fulkerson, D. R.},
title = {A Simple Algorithm for Finding Maximal Network Flows and an Application to the Hitchcock Problem},
journal = {Canadian Journal of Mathematics},
volume = {9},
```

```

pages = {210--218},
year = {1957},
note = {中文题名可译为：求最大网络流的一个简单算法及其在 Hitchcock 问题中的应用}
}

```

```

@book{ahuja1993,
author = {Ahuja, Ravindra K. and Magnanti, Thomas L. and Orlin, James B.},
title = {Network Flows: Theory, Algorithms, and Applications},
publisher = {Prentice Hall},
year = {1993},
note = {中文题名可译为：网络流：理论、算法与应用}
}

```

```

@article{hopcroftkarp1973,
author = {Hopcroft, John E. and Karp, Richard M.},
title = {An  $O(n^{5/2})$  Algorithm for Maximum Matchings in Bipartite Graphs},
journal = {SIAM Journal on Computing},
volume = {2},
number = {4},
pages = {225--231},
year = {1973},
note = {中文题名可译为：二分图最大匹配的  $O(n^{5/2})$  算法}
}

```

```

@article{brelaz1979,
author = {Br{\e}laz, Daniel},
title = {New Methods to Color the Vertices of a Graph},
journal = {Communications of the ACM},
volume = {22},
number = {4},
pages = {251--256},
year = {1979},
note = {中文题名可译为：图顶点着色的新方法}
}

```

```

@article{johnson1977,
author = {Johnson, Donald B.},
title = {Efficient Algorithms for Shortest Paths in Sparse Networks},
journal = {Journal of the ACM},
volume = {24},
number = {1},
pages = {1--13},
year = {1977},
note = {中文题名可译为：稀疏网络最短路径的高效算法}
}

```

```

@article{astar1968,
author = {Hart, Peter E. and Nilsson, Nils J. and Raphael, Bertram},
title = {A Formal Basis for the Heuristic Determination of Minimum Cost Paths},
journal = {IEEE Transactions on Systems Science and Cybernetics},
volume = {4},

```

```

number = {2},
pages = {100--107},
year = {1968},
note = {中文题名可译为：启发式确定最小代价路径的形式基础}
}

```

```

@article{heldkarp1962,
author = {Held, Michael and Karp, Richard M.},
title = {A Dynamic Programming Approach to Sequencing Problems},
journal = {Journal of the Society for Industrial and Applied Mathematics},
volume = {10},
number = {1},
pages = {196--210},
year = {1962},
note = {中文题名可译为：序列问题的动态规划方法}
}

```

```

@book{holland1975,
author = {Holland, John H.},
title = {Adaptation in Natural and Artificial Systems},
publisher = {The University of Michigan Press / MIT Press edition},
year = {1975},
note = {中文题名可译为：自然与人工系统中的适应}
}

```

```

@article{kirkpatrick1983,
author = {Kirkpatrick, S. and Gelatt, C. D. and Vecchi, M. P.},
title = {Optimization by Simulated Annealing},
journal = {Science},
volume = {220},
number = {4598},
pages = {671--680},
year = {1983},
note = {中文题名可译为：通过模拟退火进行优化}
}

```

```

@article{dorigo1996,
author = {Dorigo, Marco and Maniezzo, Vittorio and Coloni, Alberto},
title = {Ant System: Optimization by a Colony of Cooperating Agents},
journal = {IEEE Transactions on Systems, Man, and Cybernetics -- Part B},
volume = {26},
number = {1},
pages = {29--41},
year = {1996},
note = {中文题名可译为：蚁群系统：由协作代理群体进行优化}
}

```

```

@article{bayermccreight1972,
author = {Bayer, Rudolf and McCreight, Edward M.},
title = {Organization and Maintenance of Large Ordered Indexes},
journal = {Acta Informatica},

```

```

volume = {1},
pages = {173--189},
year = {1972},
note = {中文题名可译为：大型有序索引的组织与维护}
}

```

```

@article{comer1979,
author = {Comer, Douglas},
title = {The Ubiquitous B-Tree},
journal = {ACM Computing Surveys},
volume = {11},
number = {2},
pages = {121--137},
year = {1979},
note = {中文题名可译为：无处不在的 B 树}
}

```

```

@inproceedings{guibassedgewick1978,
author = {Guibas, Leonidas J. and Sedgewick, Robert},
title = {A Dichromatic Framework for Balanced Trees},
booktitle = {Proceedings of the 19th Annual Symposium on Foundations of Computer Science},
pages = {8--21},
year = {1978},
note = {中文题名可译为：平衡树的双色框架}
}

```

```

@article{huffman1952,
author = {Huffman, David A.},
title = {A Method for the Construction of Minimum-Redundancy Codes},
journal = {Proceedings of the IRE},
volume = {40},
number = {9},
pages = {1098--1101},
year = {1952},
note = {中文题名可译为：构造最小冗余码的方法}
}

```

```

@online{ctex,
author = {{CTAN}},
title = {ctex: LaTeX classes and packages for Chinese typesetting},
year = {2026},
url = {https://ctan.org/pkg/ctex},
urldate = {2026-07-08},
note = {中文排版核心宏包}
}

```

```

@online{pgf,
author = {{CTAN}},
title = {pgf -- Create PostScript and PDF graphics in TeX},
year = {2026},
url = {https://ctan.org/pkg/pgf},
}

```

```

urldate = {2026-07-08},
note = {TikZ/PGF 图形系统}
}

@online{biblatexgb,
author = {{CTAN}},
title = {biblatex-gb7714-2015},
year = {2026},
url = {https://ctan.org/pkg/biblatex-gb7714-2015},
urldate = {2026-07-08},
note = {GB/T 7714 参考文献样式}
}

```

最后的判断

如果你的目标是做成一本**真正能长期维护、不断扩章的中文教材**，那么最关键的不是先把 20 多个主题一口气写满，而是先把这四件事固定下来：第一，统一的章节模板；第二，统一的图表和代码样式；第三，统一的中文参考文献工作流；第四，一个能立即编译出 PDF 的最小工程。这个骨架一旦定稳，后续补 Bellman-Ford、Floyd-Warshall、最大流、B+ 树、红黑树、ACO 等章节时，工作量会从“反复造轮子”变成“照模板填内容”。而你要求的“教材风格、严谨证明、例题、图示、Python 代码、结语、比较表”也会因此变得可持续，而不是一次性项目。 ¹⁹

¹ ⁷ ¹⁷ ¹⁹ <https://ctan.org/pkg/ctex?lang=en>
<https://ctan.org/pkg/ctex?lang=en>

² <https://www.ams.org/qam/1958-16-01/S0033-569X-1958-0102435-2/>
<https://www.ams.org/qam/1958-16-01/S0033-569X-1958-0102435-2/>

³ ⁵ ⁶ ¹¹ ¹² ¹³ New Bounds on the Complexity of the Shortest Path Problem
https://epubs.siam.org/doi/10.1137/0205006?utm_source=chatgpt.com

⁴ ⁹ <https://ctan.org/pkg/latexmk/?lang=en>
<https://ctan.org/pkg/latexmk/?lang=en>

⁸ <https://mirrors.ctan.org/macros/latex/contrib/biblatex/doc/biblatex.pdf>
<https://mirrors.ctan.org/macros/latex/contrib/biblatex/doc/biblatex.pdf>

¹⁰ <https://ctan.org/pkg/biblatex-gb7714-2015?lang=en>
<https://ctan.org/pkg/biblatex-gb7714-2015?lang=en>

¹⁴ <https://ctan.org/pkg/pgf>
<https://ctan.org/pkg/pgf>

¹⁵ <https://xlinux.nist.gov/dads/HTML/btree.html>
<https://xlinux.nist.gov/dads/HTML/btree.html>

¹⁶ <https://ctan.org/pkg/booktabs?lang=en>
<https://ctan.org/pkg/booktabs?lang=en>

¹⁸ <https://ctan.org/pkg/biber?lang=en>
<https://ctan.org/pkg/biber?lang=en>